МедТермин — Production Readiness Audit

Audit date: 2 November 2026 · Europe/Skopje
Application snapshot: `fcf2a0a` — Brighten application palette and strengthen card and navigation contrast
Repository: RisteLalkov/AppointmentsWEB
Assessment: NOT READY for an unrestricted public production launch.

1. Executive assessment

The implemented scheduling core has substantial working protections and passed the existing test suites. The audit nevertheless found **22 findings: 7 High, 9 Medium and 6 Low**. No Critical vulnerability was demonstrated. That does **not** establish that untested production infrastructure is free of critical risk.

The highest-priority problems are secure-transport enforcement under a misconfigured listener, Web-to-API rate-limit attribution, concurrent loss of all administrators, cross-clinic/control-plane failure coupling, reservation abuse, and broken account access buttons. The last issue was found only by submitting the real Razor form: direct API tests had passed.

No application fix, refactor, deployment or live database change was made. Temporary tests, fault injection, synthetic rows and backup/restore operations used newly created, isolated test databases. The deliverables are this report and sanitized evidence.

All seven High findings should be fixed or mitigated with documented acceptance before public launch. Medium findings need explicit release decisions; identity, privacy and incident-response policies must be approved by the responsible owners. A controlled internal pilot has a different exposure profile, but it does not remove these defects.

Findings at a glance

ID	Severity	Finding
H01	High	HTTPS redirection can fail open on a misconfigured production listener
H02	High	API IP limits collapse all Web visitors into the Web server’s quota
H03	High	Concurrent administrators can remove the last active administrators
H04	High	A broken unrelated clinic can prevent healthy-clinic login
H05	High	Control-plane failure interrupts clinical traffic while health still says ready
H06	High	A regular patient can reserve an entire doctor day without abuse controls
H07	High	Account enable/disable buttons submit invalid boolean values
M01	Medium	Workspace bootstrap and polling grow without a data window
M02	Medium	A delayed booking failure can throw an uncaught dialog exception
M03	Medium	Stale rescheduling has no recovery action inside the booking form

ID	Severity	Finding
M04	Medium	Automated accessibility checks still find contrast and keyboard-scroll failures
M05	Medium	Public availability assumes every valid timezone has a valid midnight
M06	Medium	Sub-minute availability is accepted but slot generation rounds it down
M07	Medium	Public identity assurance and privileged-account protection need a launch policy
M08	Medium	Application audit and incident visibility do not cover important security events
M09	Medium	Privacy and terms are still placeholders for a real-data launch
L01	Low	Some malformed requests are classified as server/database outages
L02	Low	Explicit browser security policies are incomplete
L03	Low	Operational documentation contains contradictory legacy instructions
L04	Low	CI does not currently gate the full browser, accessibility or dependency-audit checks
L05	Low	Demo and PostgreSQL booking idempotency differ on the agreement flag
L06	Low	Account-creation errors discard nonsecret form input

2. Scope, access and evidence boundaries

The audit inventoried **173 tracked repository files**. Application source, views, configurations, SQL, migration/model definitions, test implementations, tooling and deployment guidance were reviewed systematically. Generated snapshots were checked through model/schema comparison and executed migrations; static media and vendored assets were inventoried and checked as assets/dependencies, not represented as manually line-reviewed third-party implementations. JSON, XML/SVG/project manifests and Python files received parse checks; all application/test JavaScript files passed syntax checks.

This is a technical review plus executed scenario audit. It is **not a claim of 100% statement/branch coverage, every possible input combination, a penetration-test certification, legal compliance certification, or production load certification**. Exact coverage and remaining gaps are listed below rather than inferred from green tests.

Access actually available

- Full repository and local build tools.
- A running **real PostgreSQL 17.11** instance in an isolated loopback Docker environment.
- Disposable clinical/platform databases, including actual migrated/provisioned schemas and restored copies.
- Local Web/API processes in both Development and Production modes; Chromium and Playwright.
- Published API/Web output, plus Windows x64 self-contained cross-publish output.

Access not available

The cloud environment could not connect to the supplied private server ports (`10.20.1.102:5432`, `10.20.1.100:5543/5544`). DNS resolution of the four medtermin.mk hosts also failed **from this environment**. This is **not evidence that those servers or domains are down or misconfigured**. No live database credentials, IIS configuration, deployed binaries, certificates, network ACLs, actual production roles/grants, monitoring or production backup archive were inspected.

Consequently, the actual live `appointments` and `appointments_platform` schema/data cannot be certified as matching GitHub. The detailed database inspection in this report refers to the actual local PostgreSQL schemas created/restored for this audit, not an assumption about the live server. See network evidence.

Environment and dependency observations

Component	Inspected/tested version or state
.NET SDK / runtime in audit environment	10.0.100 / 10.0.0
API/Web target	net10.0; nullable enabled; warnings treated as errors
EF Core / ASP.NET OpenAPI packages	10.0.12
Npgsql / Npgsql EF provider	10.0.3
Swagger UI	Swashbuckle.AspNetCore.SwaggerUI 10.3.1
PostgreSQL	17.11; real constraints and transactions exercised
Browser	Real Chromium on Linux, controlled desktop/mobile viewports
Node / Python	24.19.0 / 3.12.14
Package advisory checks	38 unique NuGet package IDs inventoried; NuGet scan and npm audit reported zero known advisories
Dependency limitation	Those scans do not certify the operating system, .NET runtime, IIS module or every vendored asset

The Windows self-contained outputs produced by this audit bundle **Microsoft.NETCore.App and Microsoft.AspNetCore.App 10.0.0**, reflecting the installed SDK. They were generated only to check packaging and were **not deployed**. Before release, rebuild with a current serviced SDK/runtime and verify the bundled versions; a clean NuGet report does not validate those runtime binaries. The actual production runtime version remains unknown.

3. Architecture and code-quality assessment

System responsibilities

Layer	Responsibility and integration
Appointments.Web	MVC/Razor entry pages, protected cookies, anti-forgery forms, same-origin `/data` proxy, JavaScript workspace and public discovery
Appointments.Api	Session authentication, clinic selection, authorization, validated operations, public projections, platform lifecycle and OpenAPI/Swagger
Appointments.Core	Scheduling, status transitions, catalogue/contact rules, reports, time conversion and appointment history snapshots
Appointments.Data	EF mappings/migrations, PostgreSQL transactions, advisory locks, optimistic concurrency, idempotency, audit rows and query scopes
Appointments.Contracts	Request/response records and public DTO projections
Shared appointments database	One schema per clinic, including original `main → public`; separate `clinic_*` schemas for future clinics

Layer	Responsibility and integration
Platform database	Company operators/sessions and clinic registry/lifecycle metadata; no clinical visit tables

The browser normally calls Web `/data`; Web forwards a protected session token to the API; the API selects a server-owned schema and applies role/domain rules before touching PostgreSQL. A company token is separate from a clinic token. Hostnames are entry points, not authorization boundaries.

Verified architectural strengths

- Booking/move/status rules are centralized and reused by the production adapter and demo, with the limited parity issue L05 noted separately.
- Server-generated schema names, explicit search paths and fully qualified binding checks prevent public-schema fallback or client-supplied connection routing.
- Clinical and platform tokens are separate, opaque and stored as hashes in their respective databases. The normal Web flow protects tokens in HttpOnly cookies rather than JavaScript/local storage.
- The production adapter uses database transactions, clinic-scoped resource locks, optimistic versions and SQL constraints together; it does not rely on client-side slot previews.
- Public endpoints use dedicated DTOs and did not expose patient/account identities in tested public responses.
- Failed mutations and stale versions did not create partial appointment/history rows in the executed tests.
- Demo access is restricted by environment/configuration; Production rejects demo endpoints and previously issued demo sessions.
- HTML escaping and Razor encoding worked for the tested hostile stored patient name. No successful SQL injection, cross-clinic record access or privilege escalation was demonstrated in the tested cases.

Maintainability and capacity concerns

The main workspace remains a roughly 1,800-line JavaScript module with repeated HTML rendering and asynchronous form lifecycles. The confirmed stale-dialog defect shows why request ownership/connected-element handling should become a shared convention. The current static/domain/adapter split is useful; a framework rewrite is not a prerequisite to fix these findings.

The identity lookup, registry refresh and discovery fan-out grow with clinic count. Schema-specific connection strings also mean distinct Npgsql pools; budget total PostgreSQL connections across clinics and API instances. The existing multi-tenant permission fixtures deliberately use non-superuser roles, but their connections disable pooling; large-scale pool exhaustion was not benchmarked here. No claim of a safe maximum tenant count is made.

4. Executed testing and coverage

Existing suites rerun at the audited snapshot

Suite	Result	What the result establishes
Release solution build	Passed; zero warnings/errors	All projects compile on the local Linux SDK
Core/domain/persistence checks	87/87	Scheduling/status/contact/catalogue/report rules and JSON persistence cases covered by that suite
MVC HTTP smoke	38/38	Real HTTP cookies, anti-forgery, roles, routes and local assets in demo mode

Suite	Result	What the result establishes
MVC discovery/company contracts	79/79	Real MVC/Razor against deterministic API stubs; not a real platform DB by itself
Chromium UI suite	140/140	Real browser layout/interactions; public API stub plus JSON workspace
Password visibility / DOM workflows	16/16 + 85/85	DOM behavior; no independent visual proof from jsdom
PostgreSQL/API/MVC integration	125/125	Two API instances, real PostgreSQL and MVC adapter, auth/revocation, conflicts, CRUD and production behavior
DOM workflows against real API	16/16 + 85/85	Web JavaScript/DOM workflows backed by the real API and PostgreSQL
Existing direct PostgreSQL constraint checks	7/7	Doctor/patient exclusion, invalid range/status, FK and role-link rejection
Legacy upgrade/repeated profile SQL	Passed	Existing records preserved through the tested migration sequence
Tenant isolation	75/75 + 12/12 DOM	Clinic binding, cross-clinic rejection, identity/persistence and old-tab protections
Platform/discovery	104/104	Real provision/resume/adopt/metadata/versioning, search, public exposure, hidden clinics and company authentication

These are assertion counts, not distinct endpoint totals or statement coverage. Some protections are intentionally exercised by more than one suite; counts should not be added into a percentage.

New audit-specific execution

Test group	Executed result
Documented API operation inventory	45 operations probed anonymously; expected public successes and protected denials recorded
Role/route matrix	180 request observations across anonymous, patient, doctor, reception and company identities
Unsupported method matrix	242/242 unsupported method requests returned 405
Entire clinical FK/CHECK inventory	15 FK + 12 CHECK rejection probes passed, with all mutations rolled back
Additional key/uniqueness and platform checks	16 clinical PK + 8 clinical unique-index probes, plus all 7 platform PK/FK/CHECK/unique cases passed and rolled back
EF model comparison	No pending model changes; all 15 EF runtime table column names/nullability match local PostgreSQL
Complete real Chromium journeys	19/19 successful checkpoints through Web → API → PostgreSQL, followed by separately recorded fault injection
Extended real forms	13 successful checkpoints before the broken access-control form; 7 independent continuation checks passed after recording that defect and using API recovery in the fixture
Account access buttons	**Failed in both directions**, H07; explicitly not counted as working because API toggles work
Accessibility	13 pages scanned; six pages had the violations detailed in M04
Paired backup/restore	Two isolated database archives restored; login, clinic binding and appointment counts verified
Published output boot	Actual published API/Web booted and served Swagger, login and CSS; assets/SQL/dependency DLLs verified
Windows x64 publish profiles	API and Web self-contained cross-publish succeeded; Windows/IIS execution remains untested

Test group	Executed result
Fault/adversarial probes	Confirmed findings are reported individually; failed probes are not hidden by the baseline passes

Initial exploratory browser selectors assumed that status dialogs close automatically and ignored hidden duplicate buttons/password toggle buttons. Those **test-harness assumptions were corrected**; they are not application findings. The reschedule conflict, dialog race, boolean form failure and other listed defects were independently observed after that distinction.

Complete functional coverage checklist

Legend: **PASS** = executed successfully for the stated cases; **ISSUE** = executed with a finding; **CODE** = inspected but not fully executed in the target environment; **UNTESTED** = not accessible or not performed.

Area/workflow	Status	Coverage and limits
Public landing, cards, navigation, motion/pause/reduced-motion	PASS	Chromium desktop/mobile, no-JS search, branding/assets; not every browser engine
Doctor/clinic discovery and Macedonian search	PASS	Broad/refined search, pagination, empty/error states, Cyrillic matches on PostgreSQL
Clinic contact card/map URLs	PASS / UNTESTED	Saved address/phone and encoded map/link construction tested; current live third-party tile service not certified
Public doctor availability	PASS / ISSUE	Day/service filters, free/empty/no-hours/staff-only cases; M05 timezone boundary
Clicked public time → login/register → selected booking	PASS	Doctor, clinic, date, time and service continuation; not a reservation until commit
Normal registration/login, lockout, password hashing	PASS / ISSUE	Actual new account, role restriction and password flow; H01/H02/M07
Login/register guards, browser Back, logout, expired/revoked cookies	PASS	Browser/history checks plus real API session expiry/revocation
Company sign-in/password change/logout	PASS	Separate policy/token and real password-change forms; privileged assurance limitations remain
Reception, doctor and patient navigation/permissions	PASS	Positive and negative server/DOM/browser paths; not authorization by hostname
Doctor/specialty/service creation/edit/assignment	PASS	Actual forms and adapter tests, version checks, inactive catalogue behavior
Doctor menu, list/card preferences, simple list-only catalogues	PASS	Desktop/touch/keyboard and narrow-layout checks
Patient directory/contact/profile/history	PASS	Staff edits/version conflict, patient self-only access, scoped visit history, hostile-name encoding
Patient/doctor/reception linked-account creation	PASS	Three actual role/form paths and profile filtering
Account credential reset/session revocation	PASS	Actual receptionist form and another browser session
Account enable/disable UI	ISSUE	H07 invalid Razor boolean payload; API behavior works separately
Last administrator / simultaneous staff administration	ISSUE	H03 deterministic race

Area/workflow	Status	Coverage and limits
Weekly hours, split periods, exceptions, duplicate prevention/removal	PASS / ISSUE	Real forms/API/SQL; M06 precision mismatch
Original worksheet notes and weekday ordering	PASS	UI hides source instructions; weekday sorting and independent Wednesday availability tested
Booking, agreed time, service duration and idempotency	PASS / ISSUE	Ordinary/staff-agreed paths and concurrency; H06 abuse policy and L05 demo parity
Doctor/patient double booking and adjacent intervals	PASS	Two-instance races, SQL GiST exclusions, patient conflict across doctors
Reschedule and drag/drop behavior	PASS / ISSUE	DOM/HTTP/actual patient rescheduling; stale recovery M03; physical pointer drag across every browser not performed
Confirm/cancel/complete/no-show and overdue pending resolution	PASS	Core/API status cases; real confirm/cancel, atomic history and stale versions
Confirmation dialogs, password toggles and booking searches	PASS / ISSUE	Narrow viewport and keyboard/touch checks; M02 delayed dialog replacement
Calendar views, filters, ranges and availability overlays	PASS / ISSUE	Browser/DOM integration and real page; M04 keyboard/contrast and growth concerns
Reports/filtering/metrics/export	PASS / CODE	Domain/API and DOM CSV escaping checks; current-capacity estimates documented; every spreadsheet application not executed
Clinic provisioning/retry/adoption/metadata/public flags	PASS	Real PostgreSQL/platform suites; actual company create and first new-clinic login
Platform/clinical outage handling and readiness	ISSUE	H04/H05 reproduced; broader infrastructure chaos testing not performed
JSON body/method/enum/length/version/ownership rejection	PASS / ISSUE	Matrix and targeted negatives; L01 status codes, M06 precision
CSRF, cookie flags, open return URLs, public data projection	PASS / CODE	Tested protections; actual ingress cookies/TLS/proxy policy still need verification
Database installation/upgrade/constraints/schema binding	PASS	Actual local schema and data inspected, not just migrations
Backup and paired restore	PASS / UNTESTED	Isolated restore rehearsal; actual production archives/owners/grants/PITR untested
Larger datasets/performance	ISSUE / UNTESTED	M01 single-client synthetic sample; full representative multi-user soak/stress untested
Dependency advisory and asset/manifests checks	PASS / CODE	No reported advisories; not a substitute for runtime/OS/vendor servicing
Windows/IIS/DNS/certificates/production database	UNTESTED	Cross-publish only; live endpoints inaccessible from audit environment
Accessibility	ISSUE / UNTESTED	Axe + browser keyboard cases; no complete assistive-technology/user study

5. Findings and recommended fixes

Severity reflects potential impact and release priority. Some findings are demonstrated defects; others are explicitly labeled architectural, assurance or launch-content gaps. A missing optional feature is not automatically treated as an exploitable vulnerability.

H01 · High — HTTPS redirection can fail open on a misconfigured production listener

Evidence: Reproduced using the actual published API output in Production with Hosting:AllowHttpForTesting=false. An HTTP-only listener with no discoverable HTTPS port returned HTTP 200 and an access token for POST /api/auth/login. The log says “Failed to determine the https port for redirect.” This is conditional on ingress/listener configuration; the live IIS exposure could not be inspected.

Exact locations: src/Appointments.Api/Program.cs:78; src/Appointments.Web/Program.cs:75

Reproduction / inspection steps:

1. Boot the published API with a valid isolated database, Production, HTTP testing disabled, and only `--urls http://127.0.0.1:<port>`; do not configure an HTTPS port.
2. POST valid test credentials to /api/auth/login over HTTP.
3. Observe HTTP 200 rather than a redirect or rejection; inspect Appointments.Api-published-boot.log.

Impact: The flag alone does not establish a secure transport boundary. If an equivalent listener is reachable through production ingress, credentials and bearer tokens can travel without TLS. An IIS rule can prevent this, but that rule is not established by the repository or this test.

Recommended fix: Enforce HTTPS at public ingress, configure the real redirect target and trusted proxy scheme handling, and fail startup or reject non-HTTPS authentication requests when the intended transport guarantee cannot be established. Do not blindly trust forwarded headers. Keep loopback health exceptions explicit and narrow.

Retest / release evidence: From an external client, both Web and API HTTP entry points must redirect before login pages/credential submission or reject access. Test direct backend access, proxy forwarding, certificate trust and renewal. Do not consider a false HTTP-testing flag alone sufficient.

H02 · High — API IP limits collapse all Web visitors into the Web server’s quota

Evidence: Reproduced: 30 independent Web sign-in clients exhausted the API auth bucket; a subsequent valid login received 429. Code confirms the API keys by its TCP peer address, while the Web HttpClient supplies no original-client identity. The test clients themselves shared a loopback address; distinct public source IPs were not simulated. The 90/min discovery policy uses the same architecture (code observation).

Exact locations: src/Appointments.Api/Program.cs:54; src/Appointments.Api/Program.cs:53; src/Appointments.Web/Services/ApiClient.cs:13

Reproduction / inspection steps:

1. Use the normal Web login form from 30 separate test cookie jars within one minute. Unknown test emails suffice.
2. Immediately attempt a legitimate sign-in through the same backend.
3. Observe 429 from the API; no account lockout is necessary to trigger this.

Impact: One visitor or a normal burst of visitors can block unrelated clinic/company sign-ins. Public search has the analogous shared-IP limit. Limits are also per API process, so scaling/recycling changes their effective budget.

Recommended fix: Apply appropriate visitor/account limits at the Web boundary and separate service-to-service budgets from public API budgets. If original-client identity is forwarded, authenticate the forwarding hop and configure explicit trusted proxies; never accept arbitrary Internet-supplied IP headers. Consider shared limiter state where multiple instances must enforce one policy. Return structured 429 responses and Retry-After.

Retest / release evidence: Load-test several genuinely distinct client addresses through the deployed topology. One client's auth/search abuse must not exhaust every visitor's quota, while same-account and same-client abuse still receive controlled rejection.

H03 · High — Concurrent administrators can remove the last active administrators

Evidence: Reproduced deterministically: two authenticated administrators attempted to disable each other. Target-account advisory locks held both requests after authentication, then were released together. Those locks were test coordination; the underlying race is between two otherwise valid overlapping API requests. Both operations returned 200 and the active-administrator count became zero.

Exact locations: src/Appointments.Api/Services/AccountService.cs:109

Reproduction / inspection steps:

1. Create an isolated clinic with exactly two enabled administrators, A and B.
2. Authenticate both. Hold the two target account locks, issue A→disable B and B→disable A, then release the locks after both requests have authenticated.
3. Query Accounts: both are disabled; both requests succeeded.

Impact: The self-disable check does not enforce a clinic-wide last-administrator invariant. A clinic can lose its administrative recovery path. The UI defect H07 currently obscures this race, but direct API access already permits it.

Recommended fix: Serialize administrator membership changes using a clinic-scoped lock, revalidate the actor where required, and refuse any change that would leave no enabled administrator. Provide an audited recovery procedure. Preserve the existing per-account locks/session revocation.

Retest / release evidence: Repeat the race across two API instances: at most one disable succeeds and at least one administrator remains enabled. Also test self-disable, repeated requests, and disabled actors awaiting locks.

H04 · High — A broken unrelated clinic can prevent healthy-clinic login

Evidence: Reproduced: change only east.Settings.ClinicId to a wrong binding in an isolated database. A valid main-clinic login then returns 503. Restore east's binding and login works again. Runtime scoped requests correctly fail closed for a misbound clinic.

Exact locations: src/Appointments.Api/Services/AutomaticLogin.cs:12; src/Appointments.Api/Services/ClinicConnections.cs:47

Reproduction / inspection steps:

1. Configure two working clinics and confirm a main-clinic login.
2. Temporarily misbind the other clinic's Settings.ClinicId, or make that schema unavailable.
3. Log into main with unchanged valid credentials. Observe 503 despite the main data remaining healthy.

Impact: Automatic login scans every clinic and opens every binding under the global identity lock. A bad schema can therefore prevent new sessions across healthy clinics. Startup also validates all enabled clinics, increasing the failure blast radius.

Recommended fix: Design an authoritative global identity-to-clinic routing index or another validated lookup that avoids synchronous full-clinic scans for every login. Preserve global uniqueness and fail-closed binding checks. Do not “fix” this by swallowing exceptions and silently ignoring identities in inaccessible clinics; define quarantine/recovery behavior explicitly.

Retest / release evidence: Fault one tenant’s schema while healthy tenants continue their permitted login/clinical workflows, with no duplicate-identity ambiguity or schema fallback. Measure login cost as clinic count increases.

H05 · High — Control-plane failure interrupts clinical traffic while health still says ready

Evidence: Reproduced: make PlatformClinics unavailable while the clinical database and an existing clinic token remain healthy. /api/bootstrap returns 503, but /health still returns 200 with status ready. Restoring the registry table restores access.

Exact locations: src/Appointments.Api/Program.cs:95; src/Appointments.Api/Services/ClinicRegistry.cs:73; src/Appointments.Api/Program.cs:114

Reproduction / inspection steps:

1. Keep a valid clinical session and a working appointments database.
2. In an isolated test, rename the platform registry table or deny its availability.
3. Call authenticated clinical bootstrap and then /health. Observe 503 versus ready/200.

Impact: Every API request synchronously refreshes the control-plane registry under a semaphore. A platform database/schema/pool fault can affect all clinics; the current health result will not identify that inability to serve traffic. This is an architectural availability risk, not a demonstrated live outage.

Recommended fix: Explicitly decide and implement the control-plane availability policy. Consider validated immutable routing snapshots/background refresh with carefully bounded stale-state behavior, or ensure a highly available control plane if fail-closed dependence is intentional. Never let caching re-enable a disabled clinic. Add readiness checks that cover required dependencies and distinguish liveness from readiness.

Retest / release evidence: Exercise platform outage, recovery, migration and pool exhaustion. The documented availability policy must hold, unknown/disabled clinics must remain blocked, and the deployment’s readiness probe must reflect actual service ability.

H06 · High — A regular patient can reserve an entire doctor day without abuse controls

Evidence: Reproduced on the published API: one ordinary self-registered patient submitted 16 non-overlapping bookings for a confirmation-required doctor. All 16 returned 200/Scheduled in 1.16 seconds and no free slots remained. No privileged role, overlap bypass, or duplicated request ID was used.

Exact locations: src/Appointments.Api/Controllers/AppointmentsController.cs:35; src/Appointments.Api/Services/AccountService.cs:60; src/Appointments.Core/SchedulingRules.cs:5

Reproduction / inspection steps:

1. Register a normal patient in a clinic permitting self-registration.
2. Give a doctor a free 09:00–17:00 day with 30-minute visits and confirmation required.
3. Submit each available time with a fresh request ID for that same patient.

4. All 16 pending requests reserve capacity. Existing overlap protection correctly allows these adjacent appointments.

Impact: The system lacks a configurable active/future booking cap, a pending-request abuse policy, or verification barrier for public sign-ups. Valid requests can monopolize availability, including pending requests that staff have not approved.

Recommended fix: Agree a clinic-configurable patient booking policy: active/future and per-doctor/day limits, abuse-aware rate controls, verified onboarding, and an explicit pending-request expiry/review policy. Allow audited staff exceptions. Do not automatically expire confirmed appointments without an approved business policy. Disabling public registration is a possible temporary mitigation if staff verify new users.

Retest / release evidence: Repeat the full-day reservation attempt: a documented limit must stop abuse without breaking legitimate multiple visits or authorized staff bookings. Test expiry, cancellation, staff overrides and concurrent quota checks.

H07 · High — Account enable/disable buttons submit invalid boolean values

Evidence: Reproduced in real Chromium against Web → API → PostgreSQL. For an enabled account the hidden value attribute is omitted and the form posts enabled=. For a disabled account Razor renders value="value" and the form posts enabled=value. Both requests show the generic validation error and leave access unchanged. Explicit JSON API toggles succeed.

Exact locations: src/Appointments.Web/Views/Accounts/Index.cshtml:27; src/Appointments.Web/Controllers/AccountsController.cs:23

Reproduction / inspection steps:

1. As reception, open Accounts & access and click disable on another enabled account.
2. Inspect the POST payload: enabled is empty; the account remains enabled.
3. Disable that account using an authorized explicit API request, then click enable in the Web UI.
4. The Web form now posts enabled=value and still fails.

Impact: Normal staff cannot revoke or restore account access from the intended screen, including during a compromised-account response. Password reset revocation and direct API access are separate working paths, not a replacement for a functioning access control.

Recommended fix: Render an explicit string value, for example value="@(a.Enabled ? "false" : "true")", or use a correctly bound view model/tag helper. Add real-form round-trip tests for both directions, session revocation and the last-administrator invariant H03. Improve the access-form error message so it does not incorrectly mention passwords.

Retest / release evidence: Both Web buttons must post literal true/false, update the database, show the correct notice, and immediately revoke affected sessions when required. No API-only shortcut should be used for the UI acceptance test.

M01 · Medium — Workspace bootstrap and polling grow without a data window

Evidence: Reproduced with 5,003 synthetic patients and 30,002 appointments: bootstrap returned every visible row; the re-serialized JSON representation was about 11.99 MB. That local request took about 559 ms versus a small-clinic median around 16 ms. This was a synthetic single-client sample, not a production load/SLA certification.

Exact locations: src/Appointments.Data/PostgresAppointments.cs:14; src/Appointments.Web/wwwroot/js/app.js:1768

Reproduction / inspection steps:

1. Populate only a disposable clinic with a long historical appointment dataset and thousands of contacts.
2. Request /api/bootstrap as reception.
3. Leave an idle workspace open and observe full-bootstrap polling every 20 seconds where polling is not paused.

Impact: History and contacts are repeatedly transferred and parsed even for a small current view. The API already has a paginated appointment list, but the UI does not use it for this refresh path. Large clinics and several simultaneous staff users amplify network, database and browser costs.

Recommended fix: Separate reference data from windowed appointment/contact queries; use the existing paginated/date-filtered API where appropriate, add version/delta refresh semantics, and page account/patient directories. Add an index/query plan and browser memory/load budget based on expected clinic scale.

Retest / release evidence: Run concurrent staff scenarios with representative multi-year data, defined latency/payload limits, measured browser responsiveness, and bounded refresh traffic. Verify role scoping on every new query.

M02 · Medium — A delayed booking failure can throw an uncaught dialog exception

Evidence: Reproduced by holding a booking response, closing the booking dialog, opening a doctor profile, and returning a controlled 503. The browser throws “Cannot set properties of null (setting innerHTML)”. The normal journeys had zero uncaught errors before this deliberate fault injection.

Exact locations: src/Appointments.Web/wwwroot/js/app.js:955; src/Appointments.Web/wwwroot/js/app.js:878

Reproduction / inspection steps:

1. Open and review a valid booking. Delay the /data/appointments response.
2. While it is pending, close that dialog and open a doctor profile, replacing dialog-content.
3. Return a 503 to the original request. Its catch handler looks up the now-absent booking-error element.

Impact: Asynchronous callbacks can write to a different or missing dialog and leave the user without a clear failure/reconciliation message. Similar unguarded catch patterns exist in other forms (code observation), but this booking case is the reproduced one.

Recommended fix: Capture a dialog/request generation and relevant element references; update only if they still belong to the same active flow and are connected. Otherwise surface a safe global notice. Preserve booking idempotency and reconcile ambiguous writes; canceling a browser request is not proof that the server did not commit.

Retest / release evidence: Delay/fail each mutable form while closing, navigating or opening another dialog. No uncaught exception or cross-dialog error should occur; ambiguous successful writes must remain discoverable without duplicate creation.

M03 · Medium — Stale rescheduling has no recovery action inside the booking form

Evidence: Reproduced when a patient tab retained an older pending version while the doctor confirmed it in another session. Rescheduling returned the correct HTTP 409, preserving data, but the reschedule dialog offered no reload control. Leaving the form and refreshing/reopening details or reloading the page resolved it. Appointment details already have a history refresh control; this is not an absence of all recovery in the application.

Exact locations: src/Appointments.Web/wwwroot/js/app.js:587; src/Appointments.Web/wwwroot/js/app.js:955

Reproduction / inspection steps:

1. Keep a patient tab on a pending appointment, then confirm it in a separate doctor session.
2. Before the patient tab refreshes, open reschedule and submit another free time.
3. Observe 409 and no inline refresh/discard-and-reload action in the reschedule dialog.

Impact: Concurrency protection works, but the user can repeatedly retry stale form state without an obvious recovery path. Background polling deliberately pauses while a dialog is open.

Recommended fix: Add a clear 409 recovery action that reloads the appointment and availability, explicitly explaining whether typed choices will be retained or discarded. Reuse the existing detail refresh pattern. Never silently replace the version on a form that still shows old data.

Retest / release evidence: Run two-tab confirm/move/cancel races. A stale form must retain useful input, show a conflict, offer deliberate recovery, and never overwrite another staff/user change.

M04 · Medium — Automated accessibility checks still find contrast and keyboard-scroll failures

Evidence: Executed axe WCAG 2 A/AA and 2.1 AA scans on 13 real pages. Six pages had violations: seven page/rule instances, including 31 reported contrast nodes and one unfocusable calendar scroll region. Repeated nodes and redundant avatar initials are not 31 independent product defects; decorative initials may warrant exemption.

Exact locations: src/Appointments.Web/wwwroot/css/site.css:335; src/Appointments.Web/wwwroot/css/site.css:803; src/Appointments.Web/wwwroot/css/discover.css:69; src/Appointments.Web/wwwroot/js/app.js:1067

Reproduction / inspection steps:

1. Scan the public directory and authenticated overview, doctors, patients, calendar and reports pages.
2. Examples: inactive calendar view buttons measured 3.02:1, Scheduled status 4.15:1, directory filter chip 4.07:1, and footer text 2.62:1.
3. Tab through the calendar scrollable region; axe identifies fc-scroller-liquid-absolute as lacking keyboard focus access.

Impact: Some meaningful small text remains below 4.5:1 despite the recent theme improvement. Keyboard users may not be able to reach/scroll all calendar content reliably. Automated scans are not a full WCAG certification or screen-reader review.

Recommended fix: Adjust the actual computed foreground/background pairs, prioritizing functional controls and status labels. Make the calendar scrolling region keyboard accessible with an appropriate label/focus strategy. Review redundant decorative initials separately instead of darkening everything indiscriminately.

Retest / release evidence: Re-run automated scans and manual keyboard/zoom/screen-reader checks in target browsers. Validate focus visibility, dialogs and calendar interaction at 200% zoom and mobile sizes.

M05 · Medium — Public availability assumes every valid timezone has a valid midnight

Evidence: Reproduced for an allowed America/Havana clinic on 2027-03-14, within the tested booking horizon: public availability returned 500. That day has a nonexistent local midnight, and ToInstant returns null. Europe/Skopje scenarios did not exhibit this issue.

Exact locations: src/Appointments.Api/Services/PublicDiscovery.cs:46; src/Appointments.Core/SchedulingClock.cs:14

Reproduction / inspection steps:

1. Provision an isolated clinic with TimeZone America/Havana and a doctor available Sunday 09:00–10:00.
2. GET its public availability for 2027-03-14.
3. The forced nullable Value access for the local day boundary throws.

Impact: The API accepts timezone configurations that its day-window construction cannot safely handle. This is a conditional bug for supported zones, not evidence of incorrect Skopje booking times.

Recommended fix: Build valid local-day query bounds without assuming midnight exists, or explicitly restrict supported zones/configurations with validation. Keep the existing policy of rejecting ambiguous/nonexistent actual appointment times; do not silently shift a chosen appointment.

Retest / release evidence: Test midnight DST gaps/repeats, ordinary Skopje DST, supported international zones and boundary dates. The endpoint must return valid information or a documented validation response, never 500.

M06 · Medium — Sub-minute availability is accepted but slot generation rounds it down

Evidence: Reproduced: a weekly start of 09:00:00.5000000 was accepted with HTTP 200, but the first returned slot was 09:00:00, before the configured start. The schedule validator checks Second, not fractional ticks. Additional availability periods have the analogous precision inconsistency (code observation).

Exact locations: src/Appointments.Core/AppointmentService.cs:187; src/Appointments.Core/SchedulingRules.cs:16

Reproduction / inspection steps:

1. Save a valid doctor schedule whose start/end include a fractional second with Second still equal to zero.
2. Request slots for the matching weekday.
3. Observe a rounded-down first start outside the saved interval.

Impact: The API and scheduling grid disagree about accepted time precision. Normal minute-only browser inputs reduce exposure, but direct/API clients can create inconsistent availability and later schedule-protection conflicts.

Recommended fix: Enforce whole-minute ticks consistently for schedules, exceptions and all relevant DTOs, or implement a precisely specified slot quantization rule that never emits a slot outside its source interval. Keep agreed-time validation consistent.

Retest / release evidence: Test seconds and fractional seconds around start/end boundaries in weekly and one-off periods, checking that every accepted slot fits its actual saved availability.

M07 · Medium — Public identity assurance and privileged-account protection need a launch policy

Evidence: Code-confirmed and documented limitations: registration accepts an email without ownership verification and immediately enables a new patient; MFA, emailed recovery and forced first-use password change are absent. Actual registration without email delivery succeeded. Existing patient records cannot be claimed merely by matching email, which is a working protection.

Exact locations: src/Appointments.Api/Services/AccountService.cs:60; src/Appointments.Data/Entities.cs:15; README.md:95

Reproduction / inspection steps:

1. Register a fresh test email through the normal public form.
2. Sign in immediately; no email ownership proof is requested.
3. Inspect clinic/company sign-in and first-use credential flows for MFA or forced rotation; neither is implemented.

Impact: An unverified identity can reserve a globally unique email and obtain booking access. Privileged accounts rely on passwords and lockout alone. This is an assurance/policy gap, not a demonstrated takeover of an existing clinical profile.

Recommended fix: Decide verified public onboarding, account recovery, privileged MFA or restricted administrative ingress, and staff identity checks before launch. If integrations are not ready, disable public registration where appropriate and use verified staff-created accounts. Do not imply email/phone verification or notifications in product claims.

Retest / release evidence: Approve and test the chosen ownership/recovery policy, failed-verification paths, MFA or compensating controls, secure initial credential delivery and session revocation.

M08 · Medium — Application audit and incident visibility do not cover important security events

Evidence: Code observation supported by the exported audit schema: the application records selected mutations but not an authentication-failure event history, access-denied/read-access trail, or a full company metadata/password-change audit. Failed-attempt counters are reset and do not form a historical trail. The test did not inspect actual IIS, SIEM or database-provider logs.

Exact locations: src/Appointments.Api/Services/AccountService.cs:33; src/Appointments.Api/Services/PlatformService.cs:118; src/Appointments.Data/PostgresAppointments.cs:225

Reproduction / inspection steps:

1. Cause failed logins, change platform metadata and change the company password in an isolated fixture.
2. Compare the application audit model/write paths with the existing appointment/account mutation audit.
3. Identify missing event types and confirm any external logging separately.

Impact: Incident reconstruction and detection depend on infrastructure that is not established here. The in-database audit also shares the application's broad write authority and is not tamper-evident.

Recommended fix: Define a privacy-conscious event catalogue, correlation IDs, trusted client attribution and external protected log storage. Add the missing security/company events, alerts and retention/access rules without logging passwords, tokens or unnecessary patient details. Define how sensitive reads are audited if required by policy.

Retest / release evidence: Demonstrate a test incident from sign-in failure through account disable and investigation, including cross-instance correlation, alert delivery, retention and protection against accidental audit deletion.

M09 · Medium — Privacy and terms are still placeholders for a real-data launch

Evidence: Code-only launch-content gap: the public footer marks Privacy and Terms as “Наскоро” and has no working policy links. No actual legal/compliance documentation or organizational agreement was supplied to this audit.

Exact locations: src/Appointments.Web/Views/Discover/Index.cshtml:167

Reproduction / inspection steps:

1. Open the landing-page footer.
2. Observe the disabled privacy/terms text instead of a published notice.

Impact: Visitors are about to supply personal contact and appointment data without the intended public policy pages being available. The appropriate legal obligations depend on the deployment and organization; this audit is not a legal compliance certification.

Recommended fix: Have the responsible company/clinic owners approve and publish accurate privacy, terms, retention/deletion, identity-verification and support policies. Address external map requests and relevant processing/vendor responsibilities. Link them from registration and the footer as required by the approved policy.

Retest / release evidence: Confirm the final documents and operational procedures with the responsible owners, verify working accessible links, and ensure the product's actual behavior matches its claims.

L01 · Low — Some malformed requests are classified as server/database outages

Evidence: Reproduced: a body over the 64 KiB API limit returned 500 rather than 413. A booking with empty serviceId on a legacy/no-service doctor reached a foreign-key failure and returned 503; the rules had treated empty as absent. Neither probe created an invalid appointment.

Exact locations: src/Appointments.Api/Program.cs:67; src/Appointments.Core/Catalogue.cs:91; src/Appointments.Core/AppointmentService.cs:86

Reproduction / inspection steps:

1. POST a roughly 70 KiB JSON body to an API endpoint.
2. POST a valid legacy-doctor booking with serviceId="" rather than null.
3. Observe 500 and 503 respectively instead of precise input/size handling.

Impact: These responses mislead users, monitoring and retry logic into diagnosing application/database failure for invalid input. Database integrity remains protected.

Recommended fix: Preserve BadHttpRequestException status codes such as 413, normalize optional service IDs consistently or reject empty IDs, and return structured validation errors. Include structured rate-limit responses as part of H02.

Retest / release evidence: Assert exact 400/409/413/429/503 semantics for malformed, stale, oversized, limited and unavailable cases, with no partial persistence.

L02 · Low — Explicit browser security policies are incomplete

Evidence: Observed on the real workspace response: no Content-Security-Policy or X-Frame-Options header. X-Content-Type-Options and Referrer-Policy were present. No successful clickjacking or XSS exploit was demonstrated; HttpOnly/SameSite cookies, CSRF validation and HTML escaping remain important working protections.

Exact locations: src/Appointments.Web/Program.cs:80; src/Appointments.Web/Views/Shared/_Layout.cshtml:15

Reproduction / inspection steps:

1. Inspect security headers on authenticated Web pages and their actual deployed ingress.
2. Compare script, frame, image and map dependencies before choosing policies.

Impact: The browser has less defense in depth against unintended framing and future injection than it could. Infrastructure may add headers, but that was not verifiable here.

Recommended fix: Introduce a tested CSP, beginning in report-only mode if appropriate, including frame-ancestors and suitable script/connect/frame sources. Consider a Permissions-Policy and consistent frame protection. Preserve approved Google Maps embedding and existing local assets.

Retest / release evidence: Check real ingress headers and normal workflows with enforcement enabled; verify that unauthorized framing is blocked without breaking intended embedded content.

L03 · Low — Operational documentation contains contradictory legacy instructions

Evidence: Code/document review: README still states that staff cannot complete an unconfirmed past request, although the implemented/tested overdue-resolution path permits completion/no-show after its end. Some older feature documents still instruct migration per separate clinic database. The printed system guide is explicitly a historical snapshot and should not be mistaken for current instructions.

Exact locations: README.md:140; docs/PATIENT_PROFILES.md:31; docs/SHARED_DATABASE.md:1

Reproduction / inspection steps:

1. Compare README appointment-status guidance with ChangeStatus and the overdue tests.
2. Compare older per-clinic database migration notes with current shared-database/schema instructions.

Impact: An operator can follow an obsolete mental model during deployment or support, particularly when managing the original public clinic and newer schemas.

Recommended fix: Consolidate a versioned current deployment/runbook index, mark superseded documents clearly, and update current status/migration guidance. Preserve historical artifacts as labeled snapshots rather than silently treating them as current.

Retest / release evidence: Have a second operator follow only the current runbook through a disposable deployment, upgrade, clinic registration, backup and recovery.

L04 · Low — CI does not currently gate the full browser, accessibility or dependency-audit checks

Evidence: Workflow code observation: discovery_web_smoke runs Chromium only when TEST_BROWSER_EXECUTABLE is set; the checked-in CI job does not install/configure that browser path. DOM checks are present. Axe and package vulnerability audits are not configured as release gates. Remote GitHub job status was not queried.

Exact locations: .github/workflows/ci.yml:1

Reproduction / inspection steps:

1. Inspect the checked-in workflow’s test environment and browser installation steps.
2. Compare it with the environment used for the 140 Chromium checks and 13 axe scans in this audit.

Impact: Passing CI does not currently imply that the browser-only failures found here would be caught before release. A green badge must not be interpreted as end-to-end production approval.

Recommended fix: Add a controlled real-browser job, accessibility regression checks, dependency-advisory checks and targeted regressions for the high-severity findings. Add Windows publish packaging verification and an actual staging smoke test where available.

Retest / release evidence: Intentionally break a representative form/keyboard workflow in a disposable branch and prove the relevant CI gate fails, then verify artifacts and redacted failure traces are retained.

L05 · Low — Demo and PostgreSQL booking idempotency differ on the agreement flag

Evidence: Reproduced with the real domain service and JSON demo store: submit a normal request, then reuse its ID with AgreedTime=true. The demo returns the original pending appointment rather than rejecting the changed request. PostgreSQL fingerprints the full canonical command and rejects changed reuse. This is a demo/testing parity defect, not a demonstrated production duplicate-booking defect.

Exact locations: src/Appointments.Core/AppointmentService.cs:60; src/Appointments.Data/PostgresAppointments.cs:73

Reproduction / inspection steps:

1. In a disposable JSON demo, book a confirmation-required doctor using a fresh request ID.
2. Reuse that ID with only AgreedTime changed.
3. Observe successful replay in demo; compare with the production adapter's fingerprint handling.

Impact: Demo tests and demonstrations can accept behavior the production backend correctly rejects, reducing confidence when relying only on demo/DOM paths.

Recommended fix: Share canonical request normalization/fingerprint semantics or explicitly align the demo duplicate comparison with all relevant booking fields and request-ID normalization. Keep the stable original-response replay behavior.

Retest / release evidence: Run the same idempotency matrix against both stores: identical retry, changed time/patient/service/agreement/duration, GUID format variants, and retries after later appointment changes.

L06 · Low — Account-creation errors discard nonsecret form input

Evidence: Reproduced through the real Web form: submit a new administrator account using an email already reserved by another account. After the validation/error redirect, name and email are both empty. Password clearing is correct and should remain.

Exact locations: src/Appointments.Web/Controllers/AccountsController.cs:25; src/Appointments.Web/Views/Accounts/Index.cshtml:11

Reproduction / inspection steps:

1. Enter a valid display name, role and password, but choose an already-used email in Accounts & access.
2. Submit and observe the server rejection.
3. The name/email/selection context is lost instead of retaining safe input for correction.

Impact: Users must re-enter data after a routine correctable error, making account provisioning more error-prone. No password exposure was observed.

Recommended fix: Use a view model or a whitelisted safe PRG state to retain name, email, role and applicable profile selection. Never retain the password in HTML, query strings, TempData or logs.

Retest / release evidence: Cause each account validation/uniqueness error and verify safe values remain, secrets are cleared, and linked-profile selections are revalidated before a successful retry.

6. Database assessment

Actual local structure inspected

The canonical original-clinic `public` schema contains **16 tables, 110 columns, 16 primary keys, 15 foreign keys, 12 CHECK constraints, two GiST exclusion constraints and 37 indexes**. This includes the EF migration-history table. The platform schema contains **4 tables, 28 columns, 4 primary keys, one explicit unique constraint, one foreign key, one CHECK constraint and 6 indexes**.

The final fixture also contained `clinic_east`, a newly provisioned `clinic_audit_new`, and `clinic_havana` in the **same clinical database**. Managed schemas add the single-column `CarelineProvisioning` marker to the same clinical layout. The extension objects are in `careline_extensions`. Every table/column, nullable/default/length declaration, FK/CHECK/exclusion definition and index from these fixture schemas was exported to the evidence CSVs; canonical definitions appear in the schema appendix.

All four EF migrations were applied and recorded; platform schema versions 1/2/3 were present. The model reports no pending changes. Model column names/nullability agree with the real local public schema. This is not an assertion that manual changes on the inaccessible production server are absent.

Table responsibilities

Clinical table	Purpose/integrity assessment
Accounts	Role/profile-link CHECK, unique normalized email/profile links, patient/doctor FKs; disabling and credential reset revoke sessions; H03/H07 remain
Sessions	Hashed token PK, account FK/cascade, expiry index; expired sessions rejected in execution
Patients	Name/contact lengths, unique nonempty contact email, positive optimistic version; staff contact editing is intentional
Doctors	Duration CHECK, specialty FK, separate catalogue/schedule concurrency tokens
WorkingPeriods	Owned doctor FK, day/range CHECK and doctor/day index; application additionally rejects overlaps; M06 precision
SourceSchedules	Historical import metadata, owned doctor FK; not instructions for scheduling
Specialties	Unique name and optimistic version; safe catalogue identity mapping
Services	Specialty FK, duration CHECK, unique name within specialty, version
DoctorServices	Composite PK plus doctor/service FKs; assignments are validated and serialized with catalogue writes
Appointments	Doctor/patient/service FKs, valid range/status/version checks, request uniqueness, scoped read indexes and two exclusion constraints
AppointmentHistory	Appointment FK, version uniqueness and before/after snapshot checks; existing legacy visits are not given invented history
AvailabilityExceptions	Doctor FK/date index/range CHECK; API duplicate protection relies on the doctor lock and domain rule
IdempotencyRequests	Account/request composite PK, account FK, fingerprint and stable JSON response; replay survives later appointment changes
AuditEvents	Actor/action/target/time records; limited coverage and protection discussed in M08
Settings	Clinic/timezone binding markers; fail-closed schema validation
__EFMigrationsHistory	Executed clinical migration identities

Clinical table	Purpose/integrity assessment
CarelineProvisioning (managed schemas)	One clinic marker used for safe resumable registration; not a clinical data table

Platform table	Purpose/integrity assessment
PlatformOperators	Company identity, unique email, password hash and lockout state
PlatformSessions	Hashed token PK, operator FK/cascade and expiry index
PlatformClinics	Routing/lifecycle/public metadata, state CHECK, adoption marker and optimistic version
PlatformSchemaVersions	Applied raw-SQL upgrade markers

Integrity rules actually exercised

The audit executed all 15 clinical FK rejection cases and all 12 CHECK rejection cases inside a transaction that was rolled back. The existing suite separately verified both exclusion constraints, cancelled-slot release and additional constraint cases. Runtime multi-tenant tests proved that a foreign-clinic profile cannot be linked by FK and a missing tenant table cannot fall back to public. A legacy upgrade preserved existing contacts, appointments, specialty identities and durations without inventing services/history.

All 16 canonical clinical primary keys, all eight clinical unique indexes and all seven platform primary-key/foreign-key/CHECK/unique cases also rejected targeted invalid writes. Together with the 27 clinical FK/CHECK cases, this is 58 additional constraint probes, all rolled back. Index definitions were inspected from PostgreSQL and compared with the application model/rules. Not every possible collation, manually corrupted row or concurrent direct-SQL writer permutation was generated. Application normalization and locking remain important: some invariants, such as exact exception duplicate detection, case folding and schedule overlap checks, are enforced by the service rather than by a separate SQL constraint.

Possible hardening work includes explicitly documenting which invariants direct SQL writers must preserve, adding selected duplicate/enum constraints where they serve a real operational need, separating migration/runtime privileges where feasible, and formalizing retention/archive/deletion procedures. An application database owner can modify its schemas; this design does not claim database-role isolation between all clinics or tamper-proof audit storage.

Restore rehearsal

All fixture writers were stopped before taking paired custom-format backups. Both databases restored into new isolated databases without SQL errors. A restarted API accepted the original test administrator credentials and returned the same **30,002** appointment count and original clinic binding. This proves the tested local restore path, not that the user’s actual backup files, target owners/roles/grants, encrypted secrets, production key ring or recovery-time objectives have been validated.

7. Security and business-policy decisions still required

These are explicit existing limits or release decisions, not all additional defects:

- Patient accounts belong to one clinic; email uniqueness is global. A patient from clinic A cannot simply book clinic B with that account. That behavior is enforced, but the product owner must approve it for a multi-clinic public directory.

- Doctors intentionally share the clinic contact directory and may update contacts, while seeing only their own appointment history for a selected patient. This is documented behavior, not an unauthorized cross-doctor appointment leak found by the audit.
- No email/SMS reminders, email ownership verification, self-service emailed password recovery, external calendar synchronization or medical-record functionality is implemented. The UI generally states that messages were not sent. Approve the associated manual communication/support process.
- Managed-clinic metadata supports visibility/registration flags, not a complete company-facing suspend/archive/delete lifecycle. Hiding a clinic is not the same as revoking existing clinical access. Decide whether suspension is needed before launch.
- Capacity reporting deliberately reconstructs **current** weekly hours/exceptions; historical capacity is an estimate. It should not be sold as a historical staffing audit or a billing system.
- Sessions expire after two hours; five failed password attempts temporarily lock an account for 15 minutes. Existing counters are not a full incident history. Approve operational recovery and support procedures.
- Overnight periods and ambiguous/nonexistent appointment times are intentionally constrained; do not infer support for every hospital/24-hour scheduling model.
- Original imported doctor names/flags and every clinic's actual hours/services/public contact data need owner validation before they represent a real clinic. Demo=false does not by itself disable the legacy main clinic's source-doctor import into an empty schema.

8. Production verification still outstanding

Release gate	Required evidence
Live schema and data	Read-only catalogue export and migration/version verification from the actual two databases; check ownership, all schemas, constraints/indexes, extension location, account states and legitimate clinic records
Actual database role privileges	Verify clinical/platform roles, no runtime superuser, intended schema-creation/provisioning privileges, correct restored ownership/grants and no accidental public exposure
TLS and ingress	Real DNS A/AAAA, externally trusted certificates for exact hostnames, HTTPS redirects/rejection, renewal, IIS bindings, proxy scheme/client attribution and direct-backend reachability
Runtime servicing	Current supported SDK/Hosting Bundle or self-contained runtime; inspect deployed versions, not only project package versions
Secret management	Strong rotated production/test/bootstrap credentials, safe configuration overrides/ACLs, removed unnecessary bootstrap values, no secrets in source or ordinary logs
Cookie/Data Protection durability	Persistent protected keys, correct application identity, intended multi-instance key sharing, restart/recycle behavior and cookie flags on the real HTTPS response
Deployment/rollback	Full publish output copied atomically with server settings preserved; maintenance/migration order, SQL assets, configuration validation, rollback and post-deploy smoke procedure
Backup/DR	Actual paired production archives restored with real roles/grants; coordinated consistency, off-server storage, encryption, retention, recovery-time/data-loss objectives and recovery rehearsal
Monitoring/readiness	Defined liveness/readiness, dependency alarms, authentication/abuse alerts, logs/correlation, retention, disk/pool/resource alerts and on-call response
Representative load	Expected clinic count, histories, services, concurrent receptionists, browsing load, abusive traffic, connection-pool limits, CPU/memory and latency targets
Target platforms	Actual Windows/IIS execution; Edge/Chrome on Windows, Safari/iOS, Firefox as supported; device/native-control behavior and assistive-technology checks

Release gate	Required evidence
Legal/support/business acceptance	Published policies, clinic/company responsibilities, identity proof, cancellation/confirmation/expiry rules, support/recovery and manual notification procedures

The cloud DNS/TCP failures do not resolve these gates either way. A server operator must verify them in the real environment. Do not infer live readiness from the local restore, Linux test run or Windows cross-publish alone.

9. Recommended remediation and release sequence

1. **Resolve the seven High findings first.** Start with literal boolean form values plus the backend last-administrator invariant, secure transport/ingress, trustworthy rate-limit attribution and booking-abuse policy. Treat control-plane and tenant-failure coupling as explicit availability decisions with tested mitigation.
2. Add targeted regression tests using **real forms and more than one session/API instance**, not only direct JSON requests. Preserve the successful SQL overlap, version and idempotency protections.
3. Address the remaining asynchronous UI, precision/timezone, accessibility and growth issues; approve identity, privacy and incident policies. Record any deferred Medium risk with an owner and reason.
4. Publish a staging candidate with serviced runtimes and preserved settings. Inspect the actual schemas/configuration and perform the production verification gates above, including a restore rehearsal and representative concurrent-user load.
5. Have reception, a doctor, a patient and the company operator perform documented acceptance journeys. Re-run the high-finding reproducers against the candidate; only then make a public launch decision.

Final assessment: the system is functionally substantial and its central scheduling/concurrency defenses work in the tested cases. It is **not yet approved for public production** because the reproduced defects and unverified operational controls can affect access, availability and support. This report is a concrete repair/verification plan, not a request for an application rewrite.

Appendix A. API operation coverage

Every operation in the generated OpenAPI inventory received an anonymous boundary probe. The role matrix and method matrix provide raw observations. A boundary 400/404 can be the expected result for deliberately invalid input or a disabled Production demo route; it is not proof of a complete successful business operation by itself. Positive behavior is supported by the functional suites/checklist above.

Method	Route	Anonymous probe	Additional role observations
GET	`/api/clinics`	200	anonymous: 200, patient: 200, doctor: 200, reception: 200, company: 200
GET	`/health`	200	anonymous: 200, patient: 200, doctor: 200, reception: 200, company: 200
GET	`/health/database`	401	anonymous: 401, patient: 200, doctor: 200, reception: 200, company: 401
GET	`/api/accounts`	401	anonymous: 401, patient: 403, doctor: 403, reception: 200, company: 401
POST	`/api/accounts`	401	anonymous: 401, patient: 403, doctor: 403, reception: 400, company: 401
POST	`/api/accounts/{id}/credentials`	401	anonymous: 401, patient: 403, doctor: 403, reception: 400, company: 401
POST	`/api/accounts/{id}/access`	401	anonymous: 401, patient: 403, doctor: 403, reception: 400, company: 401
GET	`/api/accounts/audit`	401	anonymous: 401, patient: 403, doctor: 403, reception: 200, company: 401
GET	`/api/reports`	401	anonymous: 401, patient: 403, doctor: 403, reception: 200, company: 401
POST	`/api/catalogue/doctors`	401	anonymous: 401, patient: 403, doctor: 403, reception: 400, company: 401
POST	`/api/catalogue/specialties`	401	anonymous: 401, patient: 403, doctor: 403, reception: 400, company: 401
POST	`/api/catalogue/services`	401	anonymous: 401, patient: 403, doctor: 403, reception: 400, company: 401
GET	`/api/bootstrap`	401	anonymous: 401, patient: 200, doctor: 200, reception: 200, company: 401
GET	`/api/appointments`	401	anonymous: 401, patient: 200, doctor: 200, reception: 200, company: 401
POST	`/api/appointments`	401	anonymous: 401, patient: 400, doctor: 400, reception: 400, company: 401
GET	`/api/appointments/{id}/history`	401	anonymous: 401, patient: 200, doctor: 200, reception: 200, company: 401
GET	`/api/slots`	401	anonymous: 401, patient: 200, doctor: 200, reception: 200, company: 401
GET	`/api/exceptions`	401	anonymous: 401, patient: 403, doctor: 200, reception: 200, company: 401

Method	Route	Anonymous probe	Additional role observations
POST	`/api/appointments/{id}/move`	401	anonymous: 401, patient: 400, doctor: 400, reception: 400, company: 401
POST	`/api/appointments/{id}/status`	401	anonymous: 401, patient: 400, doctor: 400, reception: 400, company: 401
GET	`/api/patients/{id}`	401	anonymous: 401, patient: 200, doctor: 200, reception: 200, company: 401
POST	`/api/patients/{id}`	401	anonymous: 401, patient: 400, doctor: 400, reception: 400, company: 401
POST	`/api/patients`	401	anonymous: 401, patient: 400, doctor: 400, reception: 400, company: 401
POST	`/api/schedule/{doctorId}`	401	anonymous: 401, patient: 400, doctor: 400, reception: 400, company: 401
POST	`/api/exceptions/{doctorId}`	401	anonymous: 401, patient: 400, doctor: 400, reception: 400, company: 401
POST	`/api/exceptions/{id}/remove`	401	anonymous: 401, patient: 404, doctor: 404, reception: 404, company: 401
POST	`/api/auth/register`	400	Dedicated auth/demo/functional suites; see checklist
GET	`/api/auth/me`	401	Dedicated auth/demo/functional suites; see checklist
POST	`/api/auth/logout`	401	Dedicated auth/demo/functional suites; see checklist
POST	`/api/auth/password`	401	Dedicated auth/demo/functional suites; see checklist
GET	`/api/auth/demo-users`	404	Dedicated auth/demo/functional suites; see checklist
POST	`/api/auth/demo-login`	400	Dedicated auth/demo/functional suites; see checklist
GET	`/api/public/clinics`	200	anonymous: 200, patient: 200, doctor: 200, reception: 200, company: 200
GET	`/api/public/clinics/{id}`	404	anonymous: 404, patient: 404, doctor: 404, reception: 404, company: 404
GET	`/api/public/doctors`	200	anonymous: 200, patient: 200, doctor: 200, reception: 200, company: 200
GET	`/api/public/clinics/{clinicId}/doctors/{doctorId}/availability`	200	anonymous: 200, patient: 200, doctor: 200, reception: 200, company: 200
GET	`/api/public/clinics/{clinicId}/doctors/{doctorId}`	200	anonymous: 200, patient: 200, doctor: 200, reception: 200, company: 200
POST	`/api/auth/login`	400	Dedicated auth/demo/functional suites; see checklist
POST	`/api/platform/login`	400	Dedicated auth/demo/functional suites; see checklist
GET	`/api/platform/clinics`	401	anonymous: 401, patient: 401, doctor: 401, reception: 401, company: 200
POST	`/api/platform/clinics`	401	anonymous: 401, patient: 401, doctor: 401, reception: 401, company: 400
POST	`/api/platform/clinics/{id}/adopt`	401	anonymous: 401, patient: 401, doctor: 401, reception: 401, company: 404

Method	Route	Anonymous probe	Additional role observations
POST	`/api/platform/clinics/{id}/details`	401	anonymous: 401, patient: 401, doctor: 401, reception: 401, company: 400
POST	`/api/platform/password`	401	anonymous: 401, patient: 401, doctor: 401, reception: 401, company: 400
POST	`/api/platform/logout`	401	Dedicated auth/demo/functional suites; see checklist

Appendix B. Canonical actual schema catalogue

The following is generated from local PostgreSQL catalogue queries, not reconstructed only from C# migrations. CSV exports additionally contain every equivalent fixture tenant schema. `NULL` below describes column nullability; a dash means no SQL default/length declaration in the catalogue, not an application default.

Clinical database / canonical public clinic

Accounts

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	text	NO	—	—
Email	character varying	NO	120	—
NormalizedEmail	character varying	NO	120	—
Name	character varying	NO	80	—
PasswordHash	text	NO	—	—
Role	character varying	NO	20	—
DoctorId	character varying	YES	64	—
PatientId	character varying	YES	64	—
Enabled	boolean	NO	—	—
IsDemo	boolean	NO	—	—
FailedAttempts	integer	NO	—	—
LockedUntil	timestamp with time zone	YES	—	—
CreatedAt	timestamp with time zone	NO	—	—

Constraints:

- CK_Account_RoleLink: `CHECK (((("Role")::text = 'Patient'::text) AND ("PatientId" IS NOT NULL) AND ("DoctorId" IS NULL)) OR ((("Role")::text = 'Doctor'::text) AND ("DoctorId" IS NOT NULL) AND ("PatientId" IS NULL)) OR ((("Role")::text = 'Administrator'::text) AND ("DoctorId" IS NULL) AND ("PatientId" IS NULL))))`
- FK_Accounts_Doctors_DoctorId: `FOREIGN KEY ("DoctorId") REFERENCES "Doctors"("Id") ON DELETE RESTRICT`
- FK_Accounts_Patients_PatientId: `FOREIGN KEY ("PatientId") REFERENCES "Patients"("Id") ON DELETE RESTRICT`
- PK_Accounts: `PRIMARY KEY ("Id")`

Indexes:

- IX_Accounts_DoctorId: `CREATE UNIQUE INDEX "IX_Accounts_DoctorId" ON public."Accounts" USING btree ("DoctorId")`
- IX_Accounts_NormalizedEmail: `CREATE UNIQUE INDEX "IX_Accounts_NormalizedEmail" ON public."Accounts" USING btree ("NormalizedEmail")`

- IX_Accounts_PatientId: CREATE UNIQUE INDEX "IX_Accounts_PatientId" ON public."Accounts" USING btree ("PatientId")
- PK_Accounts: CREATE UNIQUE INDEX "PK_Accounts" ON public."Accounts" USING btree ("Id")

AppointmentHistory

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	character varying	NO	64	—
AppointmentId	character varying	NO	64	—
AppointmentVersion	integer	NO	—	—
Kind	character varying	NO	20	—
ActorId	character varying	NO	64	—
ActorName	character varying	NO	200	—
ActorRole	character varying	NO	20	—
OccurredAt	timestamp with time zone	NO	—	—
BeforeStart	timestamp with time zone	YES	—	—
BeforeEnd	timestamp with time zone	YES	—	—
BeforeStatus	character varying	YES	20	—
AfterStart	timestamp with time zone	NO	—	—
AfterEnd	timestamp with time zone	NO	—	—
AfterStatus	character varying	NO	20	—

Constraints:

- CK_History_AfterRange: CHECK (("AfterStart" < "AfterEnd"))
- CK_History_Before: CHECK (((((("Kind")::text = 'Created'::text) AND ("BeforeStart" IS NULL) AND ("BeforeEnd" IS NULL) AND ("BeforeStatus" IS NULL)) OR ((("Kind")::text <> 'Created'::text) AND ("BeforeStart" IS NOT NULL) AND ("BeforeEnd" IS NOT NULL) AND ("BeforeStart" < "BeforeEnd") AND ("BeforeStatus" IS NOT NULL))))
- CK_History_Version: CHECK (("AppointmentVersion" > 0))
- FK_AppointmentHistory_Appointments_AppointmentId: FOREIGN KEY ("AppointmentId") REFERENCES "Appointments"("Id") ON DELETE RESTRICT
- PK_AppointmentHistory: PRIMARY KEY ("Id")

Indexes:

- IX_AppointmentHistory_AppointmentId_AppointmentVersion: CREATE UNIQUE INDEX "IX_AppointmentHistory_AppointmentId_AppointmentVersion" ON public."AppointmentHistory" USING btree ("AppointmentId", "AppointmentVersion")
- PK_AppointmentHistory: CREATE UNIQUE INDEX "PK_AppointmentHistory" ON public."AppointmentHistory" USING btree ("Id")

Appointments

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	character varying	NO	64	—
DoctorId	character varying	NO	64	—
PatientId	character varying	NO	64	—
Start	timestamp with time zone	NO	—	—
End	timestamp with time zone	NO	—	—
Status	character varying	NO	20	—
Version	integer	NO	—	—
CreatedBy	text	NO	—	—
UpdatedAt	timestamp with time zone	NO	—	—
RequestId	text	NO	—	—
ServiceId	text	YES	—	—
ServiceName	character varying	YES	200	—

Constraints:

- CK_Appointment_Range: CHECK (("Start" < "End"))
- CK_Appointment_Status: CHECK ((("Status")::text = ANY ((ARRAY['Scheduled'::character varying, 'Confirmed'::character varying, 'Completed'::character varying, 'Cancelled'::character varying, 'NoShow'::character varying])::text[])))
- CK_Appointment_Version: CHECK (("Version" > 0))
- EX_Appointments_Doctor: EXCLUDE USING gist ("DoctorId" WITH =, tstzrange("Start", "End", '[)'::text) WITH &&) WHERE ((("Status")::text <> 'Cancelled'::text))
- EX_Appointments_Patient: EXCLUDE USING gist ("PatientId" WITH =, tstzrange("Start", "End", '[)'::text) WITH &&) WHERE ((("Status")::text <> 'Cancelled'::text))
- FK_Appointments_Doctors_DoctorId: FOREIGN KEY ("DoctorId") REFERENCES "Doctors"("Id") ON DELETE RESTRICT
- FK_Appointments_Patients_PatientId: FOREIGN KEY ("PatientId") REFERENCES "Patients"("Id") ON DELETE RESTRICT
- FK_Appointments_Services_ServiceId: FOREIGN KEY ("ServiceId") REFERENCES "Services"("Id") ON DELETE RESTRICT
- PK_Appointments: PRIMARY KEY ("Id")

Indexes:

- EX_Appointments_Doctor: CREATE INDEX "EX_Appointments_Doctor" ON public."Appointments" USING gist ("DoctorId", tstzrange("Start", "End", '[)'::text)) WHERE (("Status")::text <> 'Cancelled'::text)
- EX_Appointments_Patient: CREATE INDEX "EX_Appointments_Patient" ON public."Appointments" USING gist ("PatientId", tstzrange("Start", "End", '[)'::text)) WHERE (("Status")::text <> 'Cancelled'::text)

- IX_Appointments_CreatedBy_RequestId: CREATE UNIQUE INDEX "IX_Appointments_CreatedBy_RequestId" ON public."Appointments" USING btree ("CreatedBy", "RequestId")
- IX_Appointments_DoctorId_Start: CREATE INDEX "IX_Appointments_DoctorId_Start" ON public."Appointments" USING btree ("DoctorId", "Start")
- IX_Appointments_PatientId_Start: CREATE INDEX "IX_Appointments_PatientId_Start" ON public."Appointments" USING btree ("PatientId", "Start")
- IX_Appointments_ServiceId: CREATE INDEX "IX_Appointments_ServiceId" ON public."Appointments" USING btree ("ServiceId")
- PK_Appointments: CREATE UNIQUE INDEX "PK_Appointments" ON public."Appointments" USING btree ("Id")

AuditEvents

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	bigint	NO	—	—
ActorId	text	NO	—	—
Action	character varying	NO	80	—
TargetId	text	NO	—	—
OccurredAt	timestamp with time zone	NO	—	—

Constraints:

- PK_AuditEvents: PRIMARY KEY ("Id")

Indexes:

- IX_AuditEvents_OccurredAt: CREATE INDEX "IX_AuditEvents_OccurredAt" ON public."AuditEvents" USING btree ("OccurredAt")
- PK_AuditEvents: CREATE UNIQUE INDEX "PK_AuditEvents" ON public."AuditEvents" USING btree ("Id")

AvailabilityExceptions

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	text	NO	—	—
DoctorId	character varying	NO	64	—
Date	date	NO	—	—
Start	time without time zone	NO	—	—
End	time without time zone	NO	—	—
IsAvailable	boolean	NO	—	—
Reason	character varying	NO	120	—

Constraints:

- CK_Exception_Range: CHECK (("Start" < "End"))

- FK_AvailabilityExceptions_Doctors_DoctorId: FOREIGN KEY ("DoctorId") REFERENCES "Doctors"("Id") ON DELETE RESTRICT
- PK_AvailabilityExceptions: PRIMARY KEY ("Id")

Indexes:

- IX_AvailabilityExceptions_DoctorId_Date: CREATE INDEX "IX_AvailabilityExceptions_DoctorId_Date" ON public."AvailabilityExceptions" USING btree ("DoctorId", "Date")
- PK_AvailabilityExceptions: CREATE UNIQUE INDEX "PK_AvailabilityExceptions" ON public."AvailabilityExceptions" USING btree ("Id")

DoctorServices

Column	PostgreSQL type	Nullable	Max length	SQL default
DoctorId	character varying	NO	64	—
ServiceId	text	NO	—	—

Constraints:

- FK_DoctorServices_Doctors_DoctorId: FOREIGN KEY ("DoctorId") REFERENCES "Doctors"("Id") ON DELETE RESTRICT
- FK_DoctorServices_Services_ServiceId: FOREIGN KEY ("ServiceId") REFERENCES "Services"("Id") ON DELETE RESTRICT
- PK_DoctorServices: PRIMARY KEY ("DoctorId", "ServiceId")

Indexes:

- IX_DoctorServices_ServiceId: CREATE INDEX "IX_DoctorServices_ServiceId" ON public."DoctorServices" USING btree ("ServiceId")
- PK_DoctorServices: CREATE UNIQUE INDEX "PK_DoctorServices" ON public."DoctorServices" USING btree ("DoctorId", "ServiceId")

Doctors

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	character varying	NO	64	—
Name	character varying	NO	200	—
Specialty	character varying	NO	200	—
BookingMethod	text	NO	—	—
Funding	text	NO	—	—
IsService	boolean	NO	—	—
RequiresConfirmation	boolean	NO	—	—
StaffOnly	boolean	NO	—	—
TuesdayFirst	boolean	NO	—	—
DurationMinutes	integer	NO	—	—

Column	PostgreSQL type	Nullable	Max length	SQL default
DemoScheduleEdited	boolean	NO	—	—
ScheduleVersion	integer	NO	—	1
CatalogVersion	integer	NO	—	1
Enabled	boolean	NO	—	true
SpecialtyId	text	YES	—	—
Subspecialty	character varying	NO	200	''::character varying

Constraints:

- CK_Doctor_Duration: CHECK ((((("DurationMinutes" >= 10) AND ("DurationMinutes" <= 120)) AND (("DurationMinutes" % 5) = 0)))
- FK_Doctors_Specialties_SpecialtyId: FOREIGN KEY ("SpecialtyId") REFERENCES "Specialties"("Id") ON DELETE RESTRICT
- PK_Doctors: PRIMARY KEY ("Id")

Indexes:

- IX_Doctors_SpecialtyId: CREATE INDEX "IX_Doctors_SpecialtyId" ON public."Doctors" USING btree ("SpecialtyId")
- PK_Doctors: CREATE UNIQUE INDEX "PK_Doctors" ON public."Doctors" USING btree ("Id")

IdempotencyRequests

Column	PostgreSQL type	Nullable	Max length	SQL default
AccountId	text	NO	—	—
RequestId	text	NO	—	—
Fingerprint	character varying	NO	64	—
ResponseJson	jsonb	NO	—	—
CreatedAt	timestamp with time zone	NO	—	—

Constraints:

- FK_IdempotencyRequests_Accounts_AccountId: FOREIGN KEY ("AccountId") REFERENCES "Accounts" ("Id") ON DELETE RESTRICT
- PK_IdempotencyRequests: PRIMARY KEY ("AccountId", "RequestId")

Indexes:

- PK_IdempotencyRequests: CREATE UNIQUE INDEX "PK_IdempotencyRequests" ON public."IdempotencyRequests" USING btree ("AccountId", "RequestId")

Patients

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	character varying	NO	64	—
Name	character varying	NO	80	—
Email	character varying	NO	120	—
Phone	character varying	NO	30	—
IsDemonstration	boolean	NO	—	—
Version	integer	NO	—	1

Constraints:

- CK_Patient_Version: CHECK (("Version" > 0))
- PK_Patients: PRIMARY KEY ("Id")

Indexes:

- IX_Patients_Email: CREATE UNIQUE INDEX "IX_Patients_Email" ON public."Patients" USING btree ("Email") WHERE (("Email")::text <> ''::text)
- PK_Patients: CREATE UNIQUE INDEX "PK_Patients" ON public."Patients" USING btree ("Id")

Services

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	text	NO	—	—
Name	character varying	NO	200	—
SpecialtyId	text	NO	—	—
DurationMinutes	integer	NO	—	—
Enabled	boolean	NO	—	—
Version	integer	NO	—	—

Constraints:

- CK_Service_Duration: CHECK ((((("DurationMinutes" >= 10) AND ("DurationMinutes" <= 120)) AND (("DurationMinutes" % 5) = 0)))
- FK_Services_Specialties_SpecialtyId: FOREIGN KEY ("SpecialtyId") REFERENCES "Specialties"("Id") ON DELETE RESTRICT
- PK_Services: PRIMARY KEY ("Id")

Indexes:

- IX_Services_SpecialtyId_Name: CREATE UNIQUE INDEX "IX_Services_SpecialtyId_Name" ON public."Services" USING btree ("SpecialtyId", "Name")
- PK_Services: CREATE UNIQUE INDEX "PK_Services" ON public."Services" USING btree ("Id")

Sessions

Column	PostgreSQL type	Nullable	Max length	SQL default
TokenHash	character varying	NO	64	—
AccountId	text	NO	—	—
ExpiresAt	timestamp with time zone	NO	—	—
IsDemo	boolean	NO	—	—

Constraints:

- FK_Sessions_Accounts_AccountId: FOREIGN KEY ("AccountId") REFERENCES "Accounts"("Id") ON DELETE CASCADE
- PK_Sessions: PRIMARY KEY ("TokenHash")

Indexes:

- IX_Sessions_AccountId: CREATE INDEX "IX_Sessions_AccountId" ON public."Sessions" USING btree ("AccountId")
- IX_Sessions_ExpiresAt: CREATE INDEX "IX_Sessions_ExpiresAt" ON public."Sessions" USING btree ("ExpiresAt")
- PK_Sessions: CREATE UNIQUE INDEX "PK_Sessions" ON public."Sessions" USING btree ("TokenHash")

Settings

Column	PostgreSQL type	Nullable	Max length	SQL default
Key	text	NO	—	—
Value	text	NO	—	—

Constraints:

- PK_Settings: PRIMARY KEY ("Key")

Indexes:

- PK_Settings: CREATE UNIQUE INDEX "PK_Settings" ON public."Settings" USING btree ("Key")

SourceSchedules

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	bigint	NO	—	—
Row	integer	NO	—	—
Days	text	NO	—	—
Start	text	NO	—	—
End	text	NO	—	—
Type	text	NO	—	—
Note	text	NO	—	—

Column	PostgreSQL type	Nullable	Max length	SQL default
DoctorId	character varying	NO	64	—

Constraints:

- FK_SourceSchedules_Doctors_DoctorId: FOREIGN KEY ("DoctorId") REFERENCES "Doctors"("Id") ON DELETE CASCADE
- PK_SourceSchedules: PRIMARY KEY ("Id")

Indexes:

- IX_SourceSchedules_DoctorId: CREATE INDEX "IX_SourceSchedules_DoctorId" ON public."SourceSchedules" USING btree ("DoctorId")
- PK_SourceSchedules: CREATE UNIQUE INDEX "PK_SourceSchedules" ON public."SourceSchedules" USING btree ("Id")

Specialties

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	text	NO	—	—
Name	character varying	NO	200	—
Version	integer	NO	—	—

Constraints:

- PK_Specialties: PRIMARY KEY ("Id")

Indexes:

- IX_Specialties_Name: CREATE UNIQUE INDEX "IX_Specialties_Name" ON public."Specialties" USING btree ("Name")
- PK_Specialties: CREATE UNIQUE INDEX "PK_Specialties" ON public."Specialties" USING btree ("Id")

WorkingPeriods

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	bigint	NO	—	—
Day	integer	NO	—	—
Start	time without time zone	NO	—	—
End	time without time zone	NO	—	—
DoctorId	character varying	NO	64	—

Constraints:

- CK_WorkingPeriod_Range: CHECK ((("Start" < "End") AND ("Day" >= 0) AND ("Day" <= 6)))
- FK_WorkingPeriods_Doctors_DoctorId: FOREIGN KEY ("DoctorId") REFERENCES "Doctors"("Id") ON DELETE CASCADE

- PK_WorkingPeriods: PRIMARY KEY ("Id")

Indexes:

- IX_WorkingPeriods_DoctorId_Day: CREATE INDEX "IX_WorkingPeriods_DoctorId_Day" ON public."WorkingPeriods" USING btree ("DoctorId", "Day")
- PK_WorkingPeriods: CREATE UNIQUE INDEX "PK_WorkingPeriods" ON public."WorkingPeriods" USING btree ("Id")

__EFMigrationsHistory

Column	PostgreSQL type	Nullable	Max length	SQL default
MigrationId	character varying	NO	150	—
ProductVersion	character varying	NO	32	—

Constraints:

- PK___EFMigrationsHistory: PRIMARY KEY ("MigrationId")

Indexes:

- PK___EFMigrationsHistory: CREATE UNIQUE INDEX "PK___EFMigrationsHistory" ON public."__EFMigrationsHistory" USING btree ("MigrationId")

Platform database / public

PlatformClinics

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	character varying	NO	40	—
Name	character varying	NO	100	—
ConnectionName	text	NO	—	—
TimeZone	character varying	NO	100	—
State	text	NO	—	—
AdminEmail	character varying	NO	120	—
AdminName	character varying	NO	80	—
Description	character varying	YES	1000	—
Address	character varying	YES	200	—
Phone	character varying	YES	30	—
Discoverable	boolean	NO	—	—
AllowRegistration	boolean	NO	—	—
CreatedBy	text	NO	—	—
CreatedAt	timestamp with time zone	NO	—	—
Adopted	boolean	NO	—	false

Column	PostgreSQL type	Nullable	Max length	SQL default
Version	integer	NO	—	1

Constraints:

- PlatformClinics_State_check: CHECK (("State" = ANY (ARRAY['Provisioning'::text, 'Ready'::text, 'Failed'::text])))
- PlatformClinics_pkey: PRIMARY KEY ("Id")

Indexes:

- PlatformClinics_pkey: CREATE UNIQUE INDEX "PlatformClinics_pkey" ON public."PlatformClinics" USING btree ("Id")

PlatformOperators

Column	PostgreSQL type	Nullable	Max length	SQL default
Id	text	NO	—	—
Email	character varying	NO	120	—
Name	character varying	NO	80	—
PasswordHash	text	NO	—	—
Enabled	boolean	NO	—	—
FailedAttempts	integer	NO	—	—
LockedUntil	timestamp with time zone	YES	—	—

Constraints:

- PlatformOperators_Email_key: UNIQUE ("Email")
- PlatformOperators_pkey: PRIMARY KEY ("Id")

Indexes:

- PlatformOperators_Email_key: CREATE UNIQUE INDEX "PlatformOperators_Email_key" ON public."PlatformOperators" USING btree ("Email")
- PlatformOperators_pkey: CREATE UNIQUE INDEX "PlatformOperators_pkey" ON public."PlatformOperators" USING btree ("Id")

PlatformSchemaVersions

Column	PostgreSQL type	Nullable	Max length	SQL default
Version	integer	NO	—	—
AppliedAt	timestamp with time zone	NO	—	now()

Constraints:

- PlatformSchemaVersions_pkey: PRIMARY KEY ("Version")

Indexes:

- `PlatformSchemaVersions_pkey`: `CREATE UNIQUE INDEX "PlatformSchemaVersions_pkey" ON public."PlatformSchemaVersions" USING btree ("Version")`

PlatformSessions

Column	PostgreSQL type	Nullable	Max length	SQL default
TokenHash	character varying	NO	64	—
OperatorId	text	NO	—	—
ExpiresAt	timestamp with time zone	NO	—	—

Constraints:

- `PlatformSessions_OperatorId_fkey`: `FOREIGN KEY ("OperatorId") REFERENCES "PlatformOperators" ("Id") ON DELETE CASCADE`
- `PlatformSessions_pkey`: `PRIMARY KEY ("TokenHash")`

Indexes:

- `IX_PlatformSessions_ExpiresAt`: `CREATE INDEX "IX_PlatformSessions_ExpiresAt" ON public."PlatformSessions" USING btree ("ExpiresAt")`
- `PlatformSessions_pkey`: `CREATE UNIQUE INDEX "PlatformSessions_pkey" ON public."PlatformSessions" USING btree ("TokenHash")`

Managed-schema provisioning marker

A newly registered managed schema additionally has `CarelineProvisioning(ClinicId text NOT NULL PRIMARY KEY)`. It is checked before resuming provisioning so that unrelated populated schemas are not silently adopted. It is not present in the original configured main/east schemas in this fixture. The clinical schema CSV includes the exact observed managed-schema entries.

Appendix C. Repository and evidence inventory

The repository inventory lists all 173 original tracked files, category, size and SHA-256 at the audited snapshot. A file inventory is not a claim of instrumented per-line test coverage. Generated migration designers/model snapshots, JSON seed data, static branding and the vendored library are explicitly distinguished from application source.

File category	Count
Application source/configuration	97
Branding/static media	3
Build, CI, configuration, tooling or repository metadata	14
Database migration/maintenance SQL	8
Documentation or historical guide	22
Generated migration/model	9
Seed/reference data	1
Test implementation/dependencies	17
Third-party vendored library/license	2

Sanitized evidence files

- `findings.json`: structured finding locations, reproduction steps, recommendations and retest criteria.
- `evidence/probes.json`, `followup.json`: the final targeted request/fault/restore/growth observations, including failed expectations.
- `endpoint-boundaries.json`, `role-matrix.json`, `method-matrix.json`: exact API request matrices.
- `accessibility.json`: page/rule/node evidence from the automated scans.
- `account-access-form.json`, `account-form-error.json`, `stale-patient-move.json`, `dialog-race.json`: reproduced UI failures.
- `demo-parity.json`: demo/production idempotency difference.
- `constraint-audit.json`, `constraint-extra.json`, `model-comparison.json`: clinical constraint probes and model/actual-column comparison.
- `clinic-*.csv`, `platform-*.csv`: complete local catalogue columns, constraints and indexes.
- `dependencies.json`, `nuget-vulnerabilities.json`, `npm-audit.json`: dependency inventories/advisory results.
- `network-check.json`, `syntax-validation.json`, `repository-inventory.json`: access limits and systematic inventory/parse evidence.

Temporary full logs, synthetic database backups and runnable exploratory scripts remain in the audit workspace. **Private fixtures, cookies, access tokens, passwords and database backups are deliberately excluded from the committed report/evidence.** Requests in the public evidence use fictional audit identities, not live patient records.

Re-running the baseline safely

Use an isolated PostgreSQL database whose name contains `test` or `integration`, never a live clinic database. The API integration suite requires a **new, empty** database; the tenancy/platform suites create their own guarded fixtures

and require the appropriate disposable-cluster administration rights. Do not copy the audit's local trust-authentication setup into production.

```
dotnet build Appointments.sln -c Release
dotnet run --project tests/Appointments.Tests -c Release --no-build
python tests/http_smoke.py
python tests/discovery_web_smoke.py
npm ci --prefix tests/frontend --ignore-scripts
npm test --prefix tests/frontend
# Set CARELINE_TEST_CONNECTION to a fresh isolated test database first:
python tests/api_integration.py --dom
# Use an isolated fixture coordinator for these guarded fixture suites:
python tests/tenancy_integration.py
python tests/platform_integration.py
```

The Chromium suite additionally needs Playwright, a browser and `TEST_BROWSER_EXECUTABLE`/`NODE_PATH` configured for that environment. The audit-specific exploit/fault cases require separate controlled fixtures and should be converted into maintained regression tests during remediation; **do not replay the destructive-looking fault injections against live databases**.